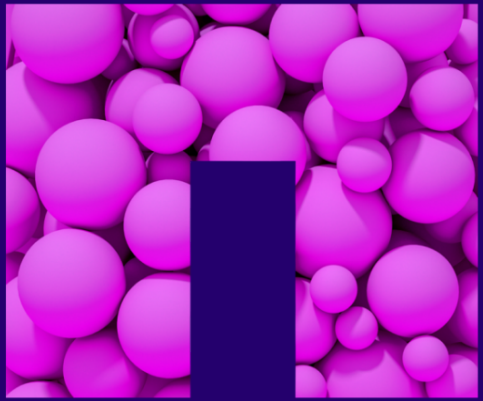
NetApp

Understanding the Magic of LLMs

A Deep Dive into the Internal Structure of LLMs



Josef Machytka <josef.machytka@netapp.com>
NetApp Deutschland GmbH
2024-10-16 - AI Workshop Mönchengladbach

Josef Machytka

- Professional Service Consultant - PostgreSQL specialist at NetApp Open Source Services (former credativ).
- 30+ years of experience with different databases.
- PostgreSQL (12y), BigQuery (7y), Oracle (15y), MySQL (12y), Elasticsearch (5y), MS SQL (5y).
- DB admin/developer, Data ingestion platforms, Data analysis, Business intelligence, Monitoring.
- Originally from Czechia, currently living in Berlin for 11 years.

- LinkedIn: www.linkedin.com/in/josef-machytka.
- ResearchGate.com: https://www.researchgate.net/profile/Josef-Machytka
- Academia.edu: https://netapp.academia.edu/JosefMachytka

Acknowledgement

- I want to thank to my colleague **Felix Alipaz-Dicke** for critical review of this presentation.
- He gave me valuable feedback and many ideas and suggestions on how to improve this talk.
- I am very grateful for his help and support.

- He is deeply interested in AI and has a lot of experience in this field.
- Do not hesitate to ask him questions or discuss with him the topic of AI.

Table of contents

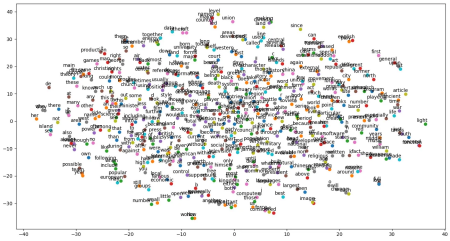
How LLMs Understand the Text

Text must be converted into numbers

- LLM is a neural network, therefore it can process only numbers.
- Text must be converted into numbers.
- But one number is not enough to reflect the meaning of the word in the text.
- Therefore multi-dimensional ”spaces of words” were created.
- Each word is in such space represented by array of numbers - vectors.
- These vectors are called ”embeddings” and they show the position of the word in the space.
- The name reflects the fact that word depends on surrounding words.
- I.e. ”is embedded” in the context.

Static Embeddings

- At the beginning of the NLP era, word embeddings were static.
- They were generated based on the frequency of words in the text.
- Word2Vec, GloVe, FastText are examples of static embeddings.
- They are still being used in many applications where the context is not that crucial.
- However they are not able to capture the meaning of the word based on the context.



(Image from the article on All about Embeddings)

Dynamic Embeddings Reflect the Meaning of Words

- ”Space of words” is a high-dimensional vector space.
- Words with similar meanings based on the context are close to each other.
- This space is created by training a neural network on a large corpus of text.
- Weights of the model defining relationships between words are adjusted during training.
- Adjusted weights are used to generate the embeddings for each word.
- These embeddings are dynamically generated based on the context of the word.

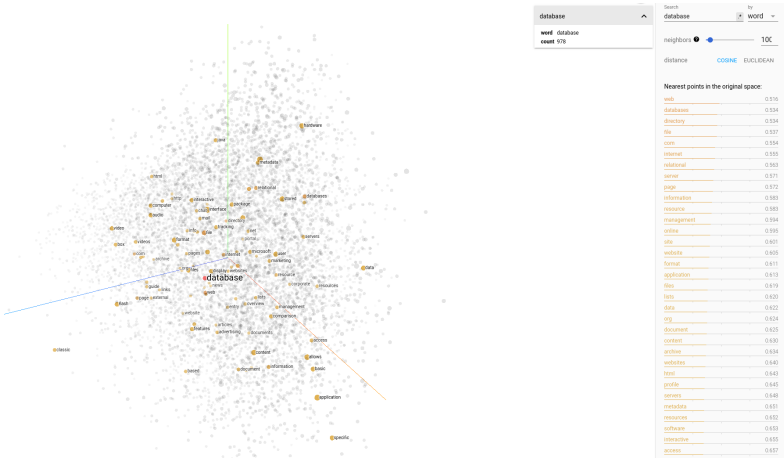
ELMo and BERT word embedding methods

- ELMo - Embeddings from Language Models
- BERT - Bidirectional Encoder Representations from Transformers

ELMo and BERT embedding method

- Allows a contextual Understanding of words (tokens).
- It was trained on the corpus of approx 30 million sentences and 1 billion words.
- It uses a bi-directional LSTM (Long Short-Term Memory) model.
- When Transformers architecture was introduced, ELMo was changed to BERT.
- BERT is using self-supervised learning, it is trained by predicting the next word in the sentence.
- Its vocabulary is approx 30,000 words.

Embeddings Projector - Word "Database"



database

word database
count 978

Search: database | by: word

neighbors: 100

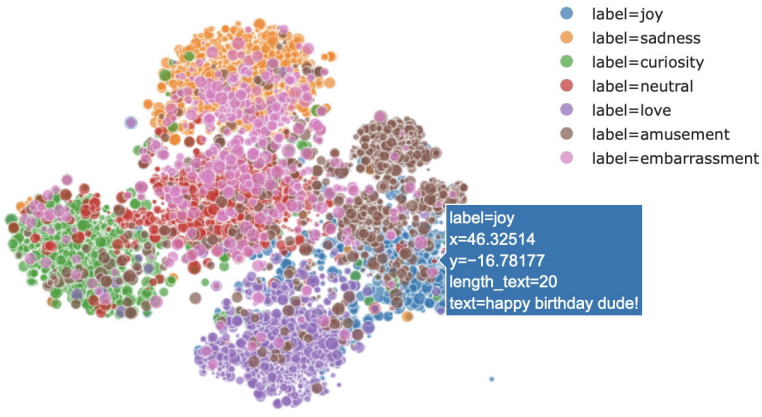
distance: COSINE | EUCLIDEAN

Nearest points in the original space:

web	0.516
databases	0.534
directory	0.534
file	0.537
com	0.554
internet	0.555
relational	0.563
server	0.571
page	0.572
information	0.583
resource	0.583
management	0.594
online	0.595
site	0.601
website	0.605
forrest	0.611
application	0.613
files	0.619
lists	0.620
data	0.622
org	0.624
document	0.625
content	0.630
archive	0.634
websites	0.640
html	0.643
profile	0.645
servers	0.648
metadata	0.651
resources	0.652
software	0.653
interactive	0.655
access	0.657

projector.tensorflow.org

Sentences Embeddings



Sentences Embedding Visualization – How to do it the Best Way

Transformers

Generative AI existed before Transformers

- Generative AI was possible even before Transformers were introduced.
- Recurrent Neural Networks (RNNs) were used for text generation.
- Google Translator, Siri or Alexa were based on RNNs.
- The Long Short-Term Memory (LSTM) networks came as an evolution of RNN.
- They were designed to process sequences of data and retain information over extended periods.
- But only Transformers with attention mechanism made possible to create modern AI models.
- Transformers are better scalable and their processing can be heavily parallelized.

Meet the Transformers

- Meet our Heroes - the Transformers! They all have AI brains!

Transformers as Neural Networks

- Transformers are a type of neural network architecture with attention mechanisms.
- Introduced in the paper ”Attention is All You Need” by Vaswani et al. in 2017.
- They are designed to handle sequential data, such as text.
- They are able to capture the meaning of the words in the context of the text.
- They look at the whole sentence at once and generate dynamic embeddings for each word.
- They analyze content of the sentence both with and without dependency on the order.

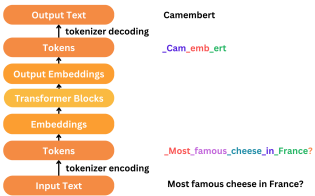
How Transformers Work

- Transformers consist of two main parts: the encoder and the decoder.
- The encoder processes the input text and generates embeddings for each word.
- The decoder generates the output text based on the embeddings generated by the encoder.
- The attention mechanism allows the model to understand importance of each word in the context.
- New models use multi-headed attention to capture different aspects of the text.
- It means that attention is calculated multiple times in parallel, each time different variant.
- Old GPT-3 had 96 attention heads running in parallel, newer models have even more.

Tokenization

Tokenization

- Tokenization is the process of breaking down text into smaller units - tokens.
- Tokens can be words, subwords, or characters, unique tokens are assigned unique IDs in the vocabulary.
- Tokenization is the first step in the LLM pipeline, because the model can only understand numbers.
- LLM produces output as a sequence of tokens, which needs to be converted back to text using the vocabulary.



(Image from the article on Tokenization)

Types of Tokenization

- Word Tokenization - splits text into words, most simple, but can struggle with unknown words.
- Subword Tokenization - splits text into smaller meaningful parts, like prefixes, stems, suffixes.
- Subword Tokenization can handle unknown words or morphologically rich languages.
- Byte Pair Encoding - splits text into common sequences of characters - GPT or Llama2 use it.
- WordPiece - similar to BPE, used in BERT model.
- SentencePiece - splits text also into subwords, but makes it even across words boundaries.

Resources

- NetAppAI GPT-4-turbo
- NetApp GitHub CoPilot AI
- Paid tier ChatGPT-4o
- Paid tier Google Gemini Advanced

- Raschka, Sebastian: Build a Large Language Model (From Scratch)
- Visualizing your own word embeddings using Tensorflow

THANK YOU

- **Questions?**

NetApp

